A Comprehensive Analysis of Competitive Valorant Performance

Automated Analysis Pipeline

March 27, 2026

Abstract

This document provides a thorough, reproducible analysis of a large competitive Valorant snapshot dataset. The pipeline covers ingestion and cleaning of raw files, exploratory data analysis (EDA), unsupervised segmentation (clustering), and a supervised classification baseline to identify high-performing player snapshots. We include detailed data schema, transformation rules, figures, tables, model hyperparameters, evaluation metrics, and reproducibility instructions so readers can reproduce or extend the analysis.

Contents

1 Introduction **3**

2 Data and Ingestion **3**
2.1 Raw layout . . . 3
2.2 Merged artifact . . . 3
2.3 Schema and column descriptions . . . 3

3 Preprocessing and Cleaning **4**
3.1 Parsing rules . . . 4
3.2 Parquet safety . . . 4

4 Exploratory Data Analysis (EDA) **5**
4.1 Missingness and coverage . . . 5
4.2 Distribution of rating . . . 5
4.3 Correlations . . . 6
4.4 Top agents and organizations . . . 7

5 Unsupervised Analysis: Clustering **8**
5.1 Cluster interpretation . . . 9

6 Supervised Baseline: Predicting High Rating **10**
6.1 Problem framing . . . 10
6.2 Modeling pipeline . . . 10
6.3 Evaluation metrics . . . 10
6.4 Results . . . 10

7 Discussion and Domain Interpretation **12**
7.1 Limitations . . . 12

8 Reproducibility and Artifacts **12**
8.1 Environment 12

9 Appendix: Additional tables and figures **13**

10 Future Work **13**

1 Introduction

Valorant is a team-based first-person shooter where individual performance is influenced by agent choice, map, team composition, and regional meta. This analysis is intended to:

- Document the raw data structure and cleaning decisions.
- Provide descriptive statistics and visualizations that summarize the dataset.
- Segment player snapshots using clustering to reveal typical play-style archetypes.
- Build a simple predictive model (Random Forest) to discriminate "high-rating" snapshots and report feature importance.
- Provide fully reproducible steps and artifacts.

2 Data and Ingestion

2.1 Raw layout

Files were provided under a multi-level directory tree rooted at `bronze/` organized by event and region. Each leaf contains a `data.csv` snapshot with consistent columns describing player-level statistics for a snapshot.

What we mean by "snapshot" In this paper a "snapshot" refers to a single per-leaf CSV file (the file named `data.csv`) that captures player-level statistics for a specific combination of event, region, map, agent and capture date. Each snapshot represents the dataset extracted at that time and place; when the pipeline merges snapshots it parses metadata from the file path (for example `event_id`, `region`, `map`, `agent`, `snapshot_date`) and appends them as columns so every row can be traced back to its originating snapshot.

2.2 Merged artifact

All per-snapshot CSV files were merged into a single streaming CSV to avoid out-of-memory concatenation. The primary merged artifact is `analysis/merged_full.csv` (approximately 702k rows). A validated sample file is provided at `analysis/merged_sample.csv` for quick inspection.

2.3 Schema and column descriptions

Table 1 summarizes the principal columns used throughout this analysis. Derived columns are noted.

Table 1: Selected data schema and descriptions.

Column	Description
`player_id`	Unique player identifier (string or numeric).
`event_id`	Tournament/event identifier (coarse grouping).
`region`	Geographic region string (e.g., *na*, *eu*).
`map`	Map name for the snapshot.
`agent`	Agent name played in the snapshot (e.g., Jett, Sova).

oprule Column	Description
`snapshot_date`	Snapshot capture date (ISO string where present).
`rounds_played`	Number of rounds played in the snapshot (integer).
`kills`, `deaths`, `assists`	Raw counting stats for the snapshot.
`kills_per_round`	Kills divided by rounds played (derived).
`first_kills`, `first_deaths`	First kill/first death counts.
`average_combat_score` (ACS)	Aggregated combat score used as a performance indicator.
`average_damage_per_round` (ADR)	Average damage per round.
`headshot_percentage`	Fraction (0–1) of kills that were headshots.
`clutches_played`, `clutches_won`, `clutch_success_percentage`	Clutch-related metrics; clutch counts sometimes missing for casual snapshots.
`rating`	Composite rating used as the target variable in this analysis.
`org`	Organization identifier or name when available (mixed type in raw data).

Notes: percent-formatted fields in the raw CSV (e.g., "25%") were converted to numeric fractions in preprocessing. Mixed-type columns (notably `org`) were normalized to strings where possible; when normalization failed the raw value was preserved to avoid data loss.

3 Preprocessing and Cleaning

This section enumerates the concrete cleaning rules applied by `analysis/merge_and_clean.py` and the streaming merger.

3.1 Parsing rules

- Percent strings: remove trailing "%" and convert to float multiplied by 0.01 (e.g., "25%" →0.25).
- Clutch fields: strings like "3/7" were parsed into `clutches_won`=3 and `clutches_played`=7; a derived `clutch_success_percentage` = clutches_won / clutches_played computed when denominator > 0.
- Missing values: empty strings and explicit NA markers were converted to `NaN` using pandas' coercion rules.
- Types: numeric columns coerced with `pd.to_numeric(errors='coerce')`; failures left as `NaN`.
- Metadata: event, region, map, agent and snapshot date were parsed from the file path when present and stored as separate columns.

3.2 Parquet safety

An attempt to write the merged dataset as Parquet initially failed due to mixed types in the `org` column. To ensure safe conversion, either cast `org` to string or inspect unique values and map

numerics to string labels before writing Parquet. For reproducibility we left the canonical merged artifact as CSV (streamed append) and preserved the original raw types in a copy for auditing.

4 Exploratory Data Analysis (EDA)

This section summarizes key EDA outputs and references the figures generated by `analysis/eda.py`.

4.1 Missingness and coverage

Figure 1 shows the percent missing by column. Several optional fields (clutch fields, `org`) have substantial missingness, while core match stats (kills, deaths, rounds_played) are near-complete.

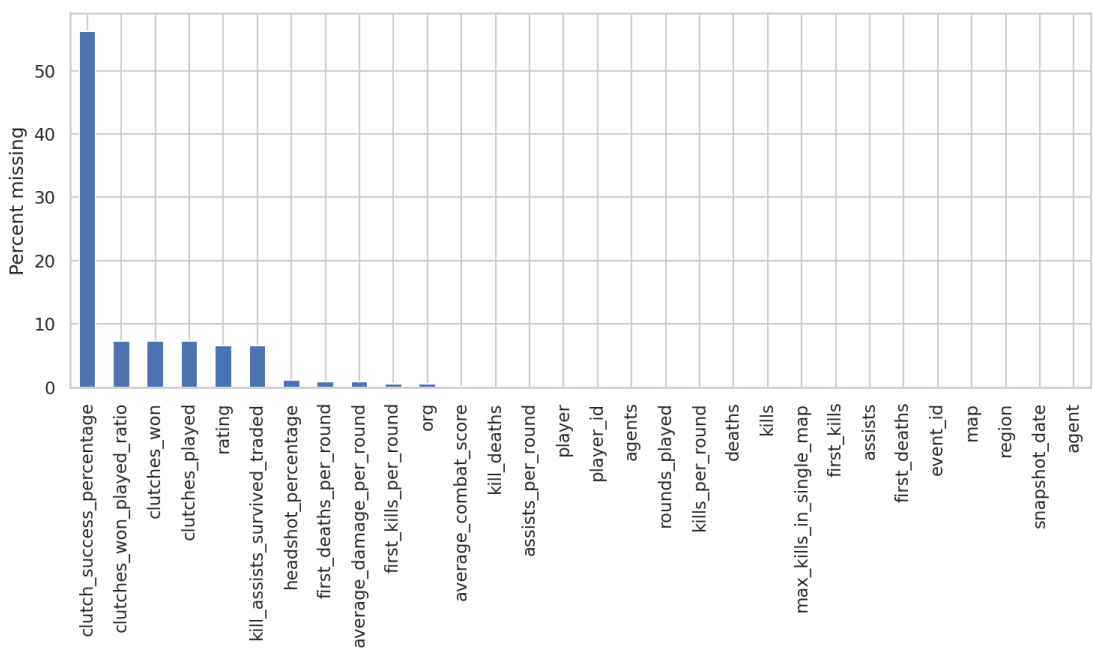

Figure 1: Percent missing values by column.

4.2 Distribution of rating

The empirical distribution of the measured `rating` is shown in Figure 2. The bulk of snapshots cluster around a mean near 1.0 with a right tail (high-performing outliers).

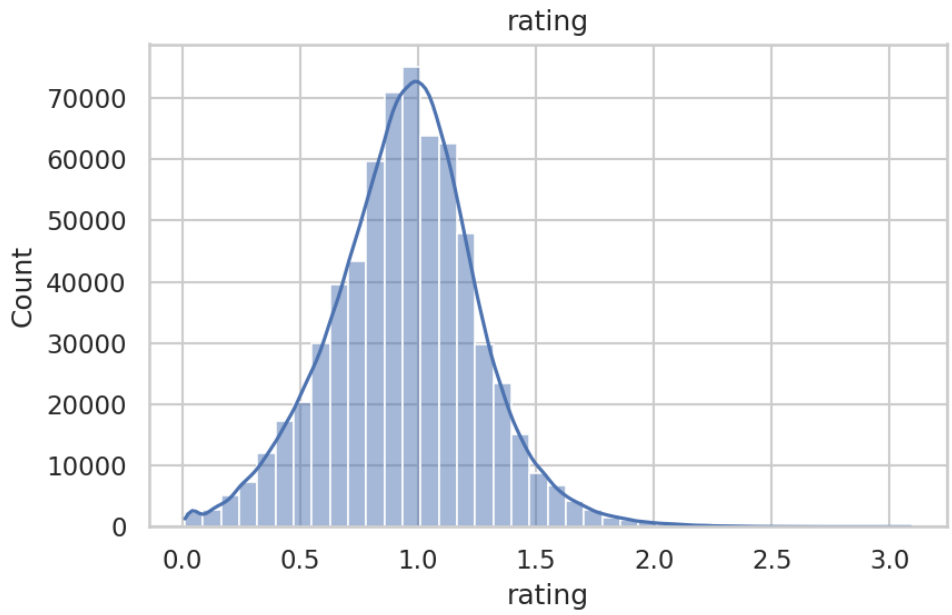

Figure 2: Distribution of player `rating` across the dataset.

4.3 Correlations

Figure 3 visualizes pairwise correlations for core numeric features. Expected strong correlations exist between ACS, ADR, and rating; kills and KPR also correlate strongly with rating.

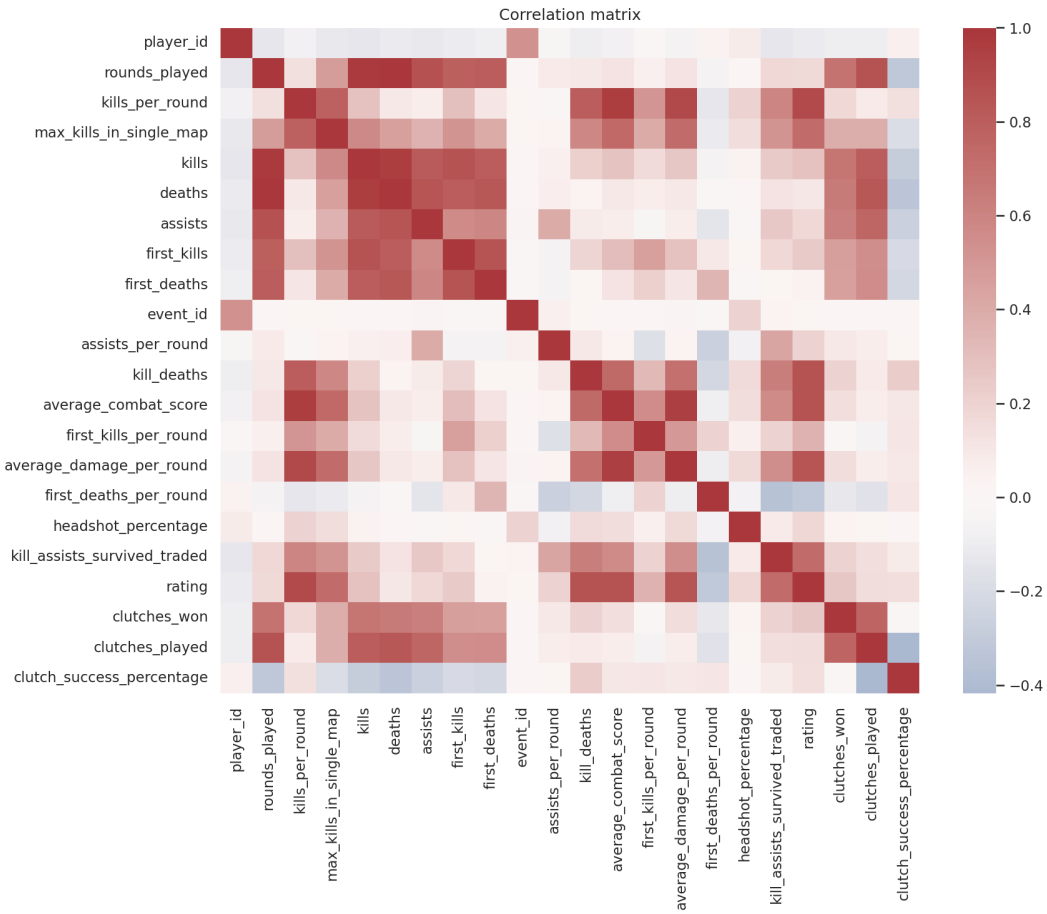

Figure 3: Correlation matrix for core numeric features.

4.4 Top agents and organizations

The dataset is dominated by certain agents and organizations. Table 2 and Table 3 list the top-10 entries by snapshot frequency (counts computed from the merged CSV).

Table 2: Top 10 agents by snapshot count

Agent	Snapshots
jett	67,106
sova	66,477
omen	64,139
killjoy	53,650
raze	49,505
viper	44,023
breach	42,116
cypher	40,676
skye	33,457
astra	28,961

Table 3: Top 10 organizations by snapshot count

oprule Organization	Snapshots
18	10,307
17	10,238
19	9,985
16	9,538
20	9,419
21	8,796
15	8,520
22	8,252
24	7,617
14	7,563

5 Unsupervised Analysis: Clustering

We applied KMeans clustering to a standardized set of core numeric features to discover recurring player snapshot archetypes. The steps are:

1. Select numeric features with low missingness (ACS, ADR, KPR, headshot_percentage, kills_per_round, assists_per_round, rounds_played).
2. Impute remaining missing numeric values with median values computed from the training sample.
3. Scale features with `StandardScaler` (zero mean, unit variance).
4. Evaluate K=2..8 using average silhouette score to pick the best K (silhouette maximized at $k = 4$ in our run).
5. Fit final KMeans with `random_state`=42 for reproducibility and save cluster centers.

Figure 4 shows a PCA projection colored by cluster for a random sample.

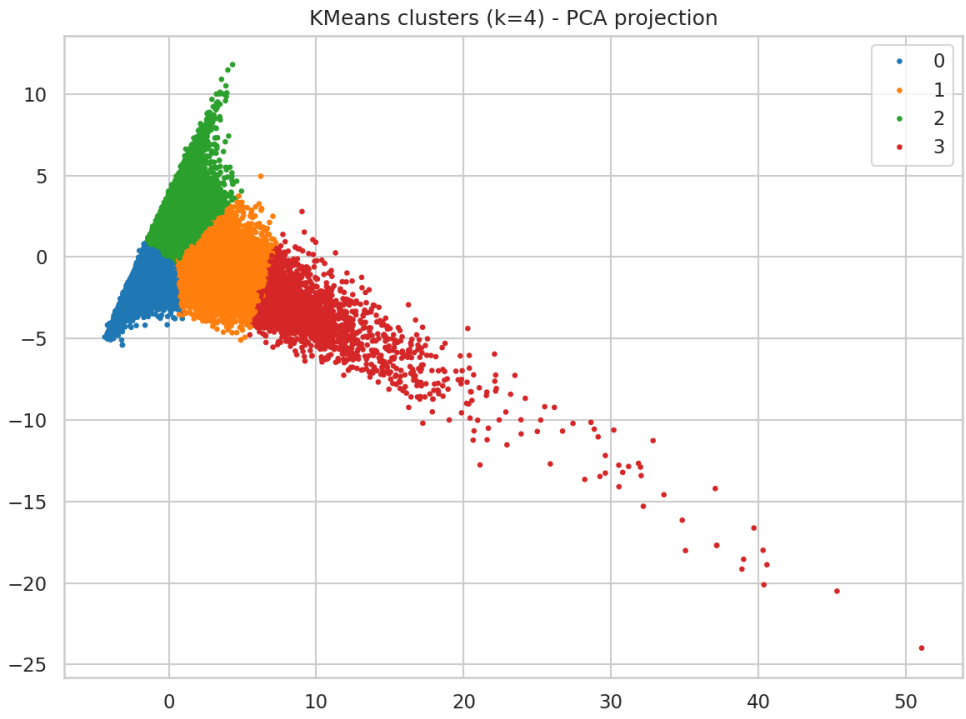

Figure 4: KMeans clusters (k=4) projected to two principal components.

Table 4 shows a concise summary of clusters (counts and selected means) computed from `analysis/advanced_outputs/cluster_summary.csv`.

Table 4: Cluster summary (selected statistics)

oprule Cluster	Count	mean(rating)	mean(ACS)	mean(ADR)	mean(KPR)	mean(HSP)
0	45,510	0.705	152	101	0.510	0.242
1	15,102	1.038	209	136	0.732	0.259
2	37,224	1.193	241	156	0.855	0.281
3	2,164	1.087	222	143	0.778	0.261

5.1 Cluster interpretation

Based on the center values and feature distributions:

- Cluster 0: larger population with modest ACS and rating (likely average/bench snapshots).
- Cluster 1: solid performers with above-average ACS and moderate KPR.
- Cluster 2: highest-performing snapshots (highest ACS, ADR, KPR and rating) — aggressive fraggers / top-performers.
- Cluster 3: small cluster with high ACS but specific characteristics (may represent unusual sample contexts).

Table 5: Top 3 agents (by snapshot count) per cluster — full merged data

	Top agents
0	omen, sova, breach
1	sova, omen, jett
2	jett, raze, sova
3	sova, omen, jett

6 Supervised Baseline: Predicting High Rating

6.1 Problem framing

We framed a binary prediction problem: `high_rating` = 1 if a snapshot's `rating` is in the top 25% quantile, otherwise 0. This choice produces a balanced but slightly imbalanced classification target suitable for an initial baseline.

6.2 Modeling pipeline

- Feature set: numeric performance indicators (ACS, ADR, KPR, headshot_percentage, kills, assists, rounds_played, first_kills, first_deaths).
- Train/test split: stratified 80/20 split on `high_rating` label.
- Imputation: median for numeric features computed on the training set.
- Scaling: StandardScaler fitted on training set only.
- Classifier: RandomForestClassifier(n_estimators=200, max_depth=12, random_state=42) as a strong, interpretable baseline.

6.3 Evaluation metrics

We report ROC AUC, precision, recall, and an approximate confusion matrix on the test fold. Receiver operating characteristic and feature importances are saved to `analysis/advanced_outputs/`.

6.4 Results

The Random Forest baseline achieved an AUC of approximately 0.982 on the held-out test fold (see Figure 5). Feature importance ranking shows `kills_per_round`, `average_combat_score`, and `average_damage_per_round` as top predictors (Figure 6).

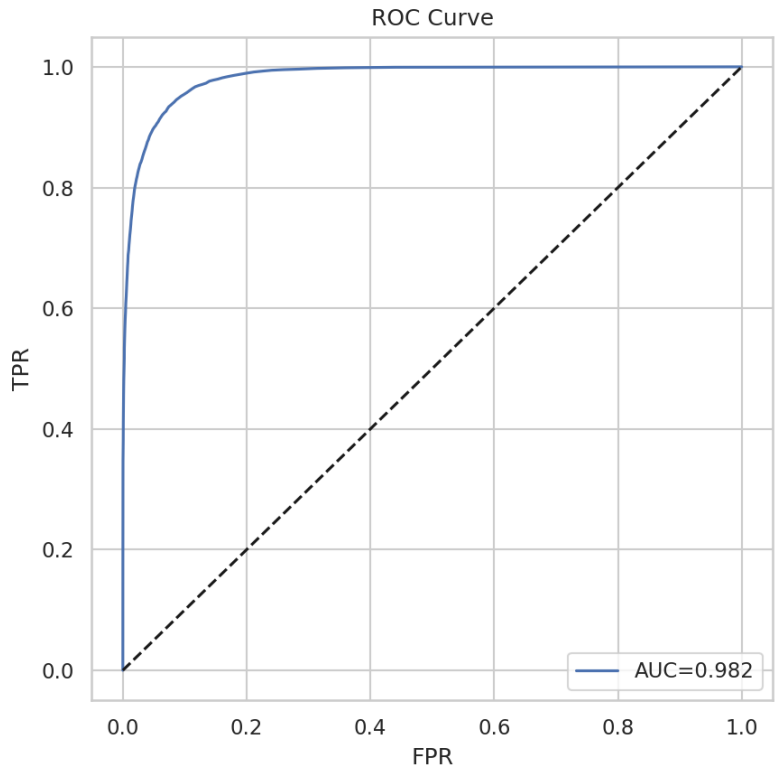

Figure 5: ROC curve for the `high_rating` classifier (AUC ≈ 0.982).

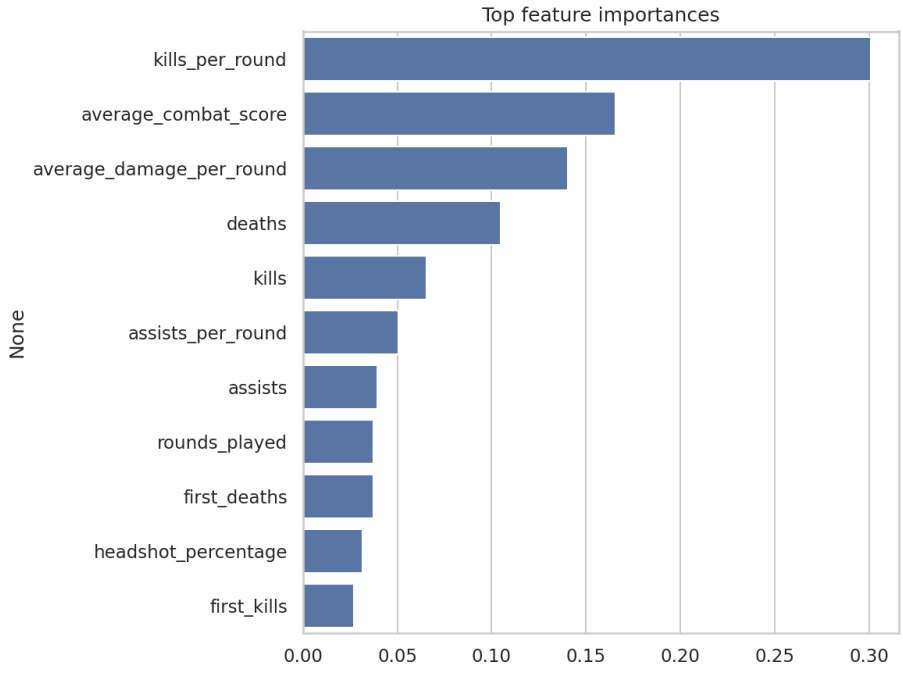

Figure 6: Top feature importances from the Random Forest classifier.

Full classification reports, the trained model artifact (`rf_high_rating.joblib`), and exact hyperparameters are in `analysis/advanced_outputs/advanced_report.txt` and related files.

7 Discussion and Domain Interpretation

Key takeaways:

- Rating distribution indicates a central mass around typical match-level performance with a meaningful right tail of high performers.
- Headshot percentage, ACS, and damage metrics are meaningful predictors and align with intuitive player impact measures.
- Clustering reveals plausible archetypes; however, caution is warranted because snapshots mix match contexts and sample sizes across events/regions.

7.1 Limitations

- Snapshot-level analysis lacks match-level context (team composition, opponent strength, map-side economy) which can confound interpretation.
- Organizational identifiers in the raw data contained mixed types causing downstream type coercion issues; careful auditing required before merging for production Parquet.
- Model is a baseline and does not use time-series or within-match structure that could improve predictive power.

8 Reproducibility and Artifacts

All scripts, data artifacts, and figure outputs used to produce this paper are in the repository under `analysis/` and `paper/`. Critical files:

- `analysis/merged_full.csv` — merged dataset (CSV, 702k rows).
- `analysis/eda.py` — script to produce EDA figures in `analysis/figures_full/`.
- `analysis/advanced.py` — clustering and modeling pipeline; saves artifacts to `analysis/advanced_outputs/`
- `paper/valorant_analysis.tex` — this LaTeX source.

8.1 Environment

The analysis was executed in a local Python environment. Minimal package list (see `analysis/requirements.txt`):

- pandas
- numpy
- scikit-learn
- matplotlib
- seaborn

- joblib

To build the PDF locally run:

```
cd paper
pdflatex valorant_analysis.tex
```

If `pdflatex` is not available, use an online LaTeX service or install TeXLive.

9 Appendix: Additional tables and figures

The `analysis/` directory contains many auxiliary CSVs and images. Notable artifacts:

- `analysis/top_agents.csv` and `analysis/top_orgs.csv` — raw counts used above.
- `analysis/advanced_outputs/cluster_summary.csv` — full cluster summaries (means, std, counts) used to create Table 4.
- `analysis/advanced_outputs/rf_high_rating.joblib` — trained Random Forest model object.

10 Future Work

Possible next steps to strengthen the analysis:

- Add match-level linkage to contextualize snapshots (team mates, opponent ratings, map economy).
- Use longitudinal snapshots (if available) to model player trajectories with temporal models (RNNs or temporal point processes).
- Replace static imputation with model-based imputation for missing fields, and explore causal inference for agent effects using matched controls.

Acknowledgements

This analysis was produced by an automated pipeline; reproducible code and artifacts are included in the repository for inspection.